

ใบงานที่ 8 Binary Search Tree

วัตถุประสงค์ เขียนโปรแกรมจำลองการทำงานของ Binary Search Tree

ทฤษฎี โหนดใด ใน Binary Search Tree ต้นไม้ย่อยทางด้านซ้ายมีค่าน้อยกว่าตัวมันเองทุกโหนด ต้นไม้ย่อยทางด้านขวา มีค่ามากกว่าตัวมันเองทุกโหนด

การทดลอง เขียนโปรแกรมจำลองการทำงาน โดยเขียนเพิ่มเติมจากใบงานที่ 7 เรื่อง Tree

1. เขียนฟังก์ชัน addB(T,Data) แล้วรันโปรแกรมทดสอบ

```
int addB(TYPE_NODEPTR T, int Data)
{
    if (T == NULL){
        //printf("Node is empty ! \n please Make a Node!");
        return MakeT(T, data);
    }

    if(data < T->data ){
        T->left = addB(T->left, data);
    }else {
        T->right = addB(T->right, data);
    }
    return T;
}
```

Main function :

```
T=NULL;
```

```
T = MakeT(T,3);InOrder(T);
```

```
printf("\n");
```

```
addB(T,7);InOrder(T);
```

```
printf("\n");
```

```
addB(T,6);InOrder(T);
```

```
printf("\n");
```

```
addB(T,2); InOrder(T);
```

```
printf("\n");
```

```
addB(T,1); InOrder(T);
```

```
printf("\n");
```

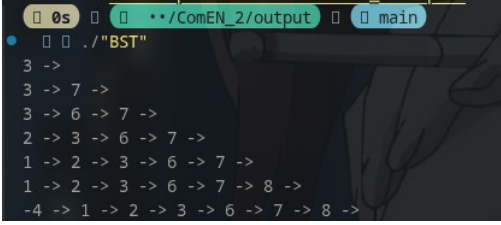
```
addB(T,8); InOrder(T);
```

```
printf("\n");
```

```
addB(T,-4); InOrder(T);
```

```
printf("\n");
```

ผลการรันโปรแกรม



```
0s  ComEN_2/output main
./"BST"
3 ->
3 -> 7 ->
3 -> 6 -> 7 ->
2 -> 3 -> 6 -> 7 ->
1 -> 2 -> 3 -> 6 -> 7 ->
1 -> 2 -> 3 -> 6 -> 7 -> 8 ->
-4 -> 1 -> 2 -> 3 -> 6 -> 7 -> 8 ->
```

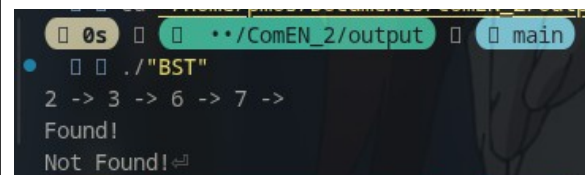
2. เขียนฟังก์ชัน ค้นหาข้อมูล ใน Binary Search Tree เพื่อค้นหาข้อมูลต้องการ

Int BstSearch(TYPE_NODEPTR T, int Data)

```
if (T == NULL){  
    printf("Not Found!");  
    return 0;  
}  
  
if (data == T->data) {  
    printf("Found!\n");  
    return 1;  
}else if (data < T->data) {  
    //printf(" | ");  
    return BstSearch(T->left, data);  
}else {  
    //printf(" | ");  
    return BstSearch(T->right, data);  
}
```

Main function :
T=NULL;
T = MakeT(T,3);
addB(T,7);
addB(T,6);
addB(T,2); inOrder(T);
printf("\n");
BstSearch(T, 6);
BstSearch(T, 5);

ผลการรันโปรแกรม



```
0s  ./ComEN_2/output  main  
./BST  
2 -> 3 -> 6 -> 7 ->  
Found!  
Not Found!
```